

Phần 2: Các Annotations JPA Cơ bản

2.1 Annotations cho Entity

2.1.1 @Entity

Đánh dấu một class là một Entity JPA. Class này sẽ được ánh xạ tới một bảng trong cơ sở dữ liệu.

Java

```
@Entity // Annotation này báo cho JPA biết rằng lớp User là một Entity và sẽ
        được ánh xạ tới một bảng trong cơ sở dữ liệu.
public class User {
    // Các thuộc tính của lớp này sẽ tương ứng với các cột trong bảng.
}
```

Giải thích: Khi một lớp được đánh dấu bằng `@Entity`, JPA sẽ tự động tạo ra một bảng trong cơ sở dữ liệu với tên mặc định là tên của lớp (trong trường hợp này là "User"). Mỗi instance của lớp này sẽ tương ứng với một hàng trong bảng đó.

2.1.2 @Table

Chỉ định tên bảng và schema cho Entity. Nếu không sử dụng, tên bảng sẽ mặc định là tên class.

Java

```
@Entity
@Table(name = "users", schema = "public") // Chỉ định rằng Entity này sẽ được
        ánh xạ tới bảng "users" trong schema "public".
public class User {
    // Các thuộc tính của lớp này sẽ tương ứng với các cột trong bảng
    "users".
}
```

Giải thích: Annotation `@Table` cho phép bạn tùy chỉnh tên bảng và schema mà Entity sẽ được ánh xạ tới. Điều này hữu ích khi tên lớp Java không khớp với tên bảng trong cơ sở dữ liệu hoặc khi bạn muốn sử dụng một schema cụ thể.

2.1.3 @Id

Đánh dấu thuộc tính là khóa chính (primary key) của Entity.

Java

```
@Entity
public class User {
    @Id // Annotation này đánh dấu trường 'id' là khóa chính của Entity User.
    private Long id; // Khóa chính phải có giá trị duy nhất cho mỗi hàng
    trong bảng.
    // Các thuộc tính khác...
}
```

Giải thích: Mỗi Entity phải có ít nhất một trường được đánh dấu bằng `@Id` để làm khóa chính. Khóa chính được sử dụng để định danh duy nhất mỗi hàng trong bảng và là cơ sở cho các thao tác tìm kiếm, cập nhật và xóa.

2.1.4 @GeneratedValue

Chỉ định cách sinh giá trị cho khóa chính tự động.

Java

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Sử dụng cơ chế tự
    tăng của cơ sở dữ liệu (ví dụ: AUTO_INCREMENT trong MySQL).
    private Long id;

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
    "user_seq") // Sử dụng một sequence được định nghĩa trong cơ sở dữ liệu.
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
    allocationSize = 1) // Định nghĩa sequence cụ thể.
    private Long id2;

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO) // JPA provider sẽ tự
    động chọn chiến lược phù hợp với cơ sở dữ liệu.
    private Long id3;

    @Id
```

```

    @GeneratedValue(strategy = GenerationType.TABLE) // Sử dụng một bảng
    riêng để quản lý các giá trị ID.
    private Long id4;
    // ...
}

```

Các strategy:

- **IDENTITY** : Sử dụng auto-increment của database (phù hợp với MySQL, SQL Server).
- **SEQUENCE** : Sử dụng sequence của database (phù hợp với Oracle, PostgreSQL).
- **TABLE** : Sử dụng bảng riêng để sinh ID (ít được dùng vì hiệu suất kém hơn).
- **AUTO** : JPA provider tự chọn strategy phù hợp (thường là SEQUENCE hoặc IDENTITY tùy thuộc vào dialect).

2.2 Annotations cho Column Mapping

2.2.1 @Column

Chỉ định thông tin chi tiết về cột trong bảng.

Java

```

@Entity
public class User {
    @Column(name = "user_name", length = 50, nullable = false, unique = true)
    // Ánh xạ trường này tới cột có tên là "user_name", độ dài tối đa 50 ký tự,
    // không được null và phải là duy nhất.
    private String name;

    @Column(precision = 10, scale = 2) // Ánh xạ tới cột số với tổng cộng 10
    // chữ số, trong đó có 2 chữ số sau dấu thập phân.
    private BigDecimal salary;

    @Column(columnDefinition = "TEXT") // Định nghĩa kiểu dữ liệu cụ thể cho
    // cột trong cơ sở dữ liệu là TEXT.
    private String description;
    // ...
}

```

Giải thích: Annotation `@Column` cung cấp khả năng tùy chỉnh chi tiết cách một trường trong Entity được ánh xạ tới một cột trong bảng cơ sở dữ liệu. Bạn có thể đặt tên cột, độ dài, khả năng null, tính duy nhất, và thậm chí cả định nghĩa kiểu dữ liệu cụ thể của cột.

2.2.2 @Temporal

Chỉ định cách ánh xạ các kiểu dữ liệu thời gian.

Java

```
@Entity
public class User {
    @Temporal(TemporalType.DATE) // Ánh xạ trường này chỉ lưu trữ ngày (ví dụ: 2023-10-26).
    private Date birthDate;

    @Temporal(TemporalType.TIME) // Ánh xạ trường này chỉ lưu trữ thời gian (ví dụ: 14:30:00).
    private Date loginTime;

    @Temporal(TemporalType.TIMESTAMP) // Ánh xạ trường này lưu trữ cả ngày và thời gian (ví dụ: 2023-10-26 14:30:00.123).
    private Date createdAt;
    // ...
}
```

Giải thích: Annotation `@Temporal` được sử dụng với các trường kiểu `java.util.Date` hoặc `java.util.Calendar` để chỉ định cách chúng được ánh xạ tới các kiểu dữ liệu ngày/giờ trong cơ sở dữ liệu. Điều này quan trọng để đảm bảo dữ liệu được lưu trữ và truy xuất đúng định dạng.

2.2.3 @Enumerated

Chỉ định cách ánh xạ enum.

Java

```
public enum Status {
    ACTIVE, INACTIVE, PENDING
}

@Entity
public class User {
```

```

    @Enumerated(EnumType.STRING) // Lưu trữ giá trị enum dưới dạng chuỗi (ví
    dụ: "ACTIVE", "INACTIVE"). Đây là cách được khuyến nghị.
    private Status status;

    @Enumerated(EnumType.ORDINAL) // Lưu trữ giá trị enum dưới dạng số nguyên
    (vị trí của enum trong khai báo, bắt đầu từ 0). Không được khuyến nghị vì dễ
    gây lỗi khi thay đổi thứ tự enum.
    private Status statusOrdinal;
    // ...
}

```

Giải thích: Annotation `@Enumerated` được sử dụng để chỉ định cách các kiểu `enum` trong Java được lưu trữ trong cơ sở dữ liệu. `EnumType.STRING` là lựa chọn an toàn hơn vì nó lưu trữ tên của enum, không bị ảnh hưởng bởi việc thay đổi thứ tự các giá trị enum.

2.2.4 @Lob

Đánh dấu thuộc tính là Large Object (BLOB hoặc CLOB).

Java

```

@Entity
public class Document {
    @Lob // Ánh xạ trường này tới một cột BLOB (Binary Large Object) trong cơ
    sở dữ liệu, dùng để lưu trữ dữ liệu nhị phân lớn như hình ảnh, video.
    private byte[] content; // BLOB

    @Lob // Ánh xạ trường này tới một cột CLOB (Character Large Object) trong
    cơ sở dữ liệu, dùng để lưu trữ dữ liệu văn bản lớn như các bài viết dài.
    private String description; // CLOB
    // ...
}

```

Giải thích: Annotation `@Lob` được sử dụng cho các trường có kích thước lớn mà không thể lưu trữ hiệu quả trong các kiểu cột thông thường. `byte[]` thường được ánh xạ tới BLOB và `String` thường được ánh xạ tới CLOB.

2.2.5 @Transient

Đánh dấu thuộc tính không được persist vào database.

Java

```

@Entity
public class User {
    private String name;

    @Transient // Annotation này báo cho JPA biết rằng trường temporaryData
    sẽ không được lưu vào cơ sở dữ liệu.
    private String temporaryData; // Không lưu vào DB
    // ...
}

```

Giải thích: Đôi khi bạn có các trường trong Entity class mà bạn không muốn chúng được ánh xạ tới các cột trong cơ sở dữ liệu (ví dụ: các trường chỉ dùng cho mục đích tính toán tạm thời hoặc hiển thị). `@Transient` giúp bạn đạt được điều này.

2.3 Annotations cho Validation

Các annotations này thuộc về Bean Validation API (JSR 380), thường được sử dụng kết hợp với JPA để đảm bảo tính hợp lệ của dữ liệu trước khi lưu vào cơ sở dữ liệu.

2.3.1 @NotNull, @NotEmpty, @NotBlank

Java

```

@Entity
public class User {
    @NotNull // Đảm bảo rằng trường này không được null.
    private String name;

    @NotEmpty // Đảm bảo rằng trường này không được null và không rỗng (empty
    string hoặc collection).
    private String email;

    @NotBlank // Đảm bảo rằng trường này không được null, không rỗng và không
    chỉ chứa khoảng trắng.
    private String username;
    // ...
}

```

Giải thích:

- `@NotNull` : Kiểm tra giá trị có phải là `null` hay không. Áp dụng cho mọi kiểu dữ liệu.

- `@NotEmpty` : Kiểm tra giá trị có phải là `null` hoặc rỗng (empty string, empty collection, empty map) hay không. Áp dụng cho `String` , `Collection` , `Map` , `Array` .
- `@NotBlank` : Kiểm tra giá trị có phải là `null` , rỗng hoặc chỉ chứa khoảng trắng hay không. Áp dụng cho `String` .

2.3.2 @Size, @Min, @Max

Java

```
@Entity
public class User {
    @Size(min = 2, max = 50) // Đảm bảo độ dài của chuỗi nằm trong khoảng từ
    2 đến 50 ký tự.
    private String name;

    @Min(18) // Đảm bảo giá trị số nguyên tối thiểu là 18.
    @Max(100) // Đảm bảo giá trị số nguyên tối đa là 100.
    private Integer age;
    // ...
}
```

Giải thích:

- `@Size` : Kiểm tra độ dài của chuỗi, kích thước của collection, map hoặc array.
- `@Min` : Kiểm tra giá trị số có lớn hơn hoặc bằng giá trị tối thiểu được chỉ định.
- `@Max` : Kiểm tra giá trị số có nhỏ hơn hoặc bằng giá trị tối đa được chỉ định.

2.3.3 @Email, @Pattern

Java

```
@Entity
public class User {
    @Email // Kiểm tra xem chuỗi có phải là một địa chỉ email hợp lệ theo
    định dạng chuẩn.
    private String email;

    @Pattern(regexp = "[0-9]{10}$") // Kiểm tra chuỗi có khớp với biểu thức
    chính quy (regex) được cung cấp.
    private String phoneNumber;
}
```

```
// ...  
}
```

Giải thích:

- `@Email` : Đảm bảo định dạng email hợp lệ.
- `@Pattern` : Cung cấp khả năng kiểm tra chuỗi dựa trên một biểu thức chính quy tùy chỉnh, rất linh hoạt cho các yêu cầu validation phức tạp.

2.4 Annotations cho Embedded Objects

2.4.1 @Embeddable và @Embedded

Cho phép nhúng một object vào Entity khác. Điều này hữu ích khi bạn muốn nhóm các thuộc tính liên quan vào một lớp riêng biệt mà không muốn tạo một bảng riêng cho lớp đó trong cơ sở dữ liệu.

Java

```
@Embeddable // Đánh dấu lớp Address là một Embeddable, có nghĩa là nó có thể  
được nhúng vào các Entity khác.  
public class Address {  
    private String street;  
    private String city;  
    private String zipCode;  
    // constructors, getters, setters  
}  
  
@Entity  
public class User {  
    @Id  
    @GeneratedValue  
    private Long id;  
  
    private String name;  
  
    @Embedded // Nhúng đối tượng Address vào Entity User. Các thuộc tính của  
Address sẽ được ánh xạ trực tiếp vào các cột của bảng User.  
    private Address address;  
  
    @Embedded  
    @AttributeOverrides({ // Sử dụng @AttributeOverrides để tùy chỉnh tên cột  
        khi có nhiều hơn một Embeddable cùng kiểu trong một Entity.
```



```

        @AttributeOverride(name = "street", column = @Column(name =
"work_street")), // Ánh xạ trường street của workAddress tới cột work_street.
        @AttributeOverride(name = "city", column = @Column(name =
"work_city")),
        @AttributeOverride(name = "zipCode", column = @Column(name =
"work_zip"))
    })
    private Address workAddress;
    // ...
}

```

Giải thích:

- `@Embeddable` : Được đặt trên một lớp để chỉ ra rằng các instance của lớp này có thể được nhúng vào các Entity khác.
- `@Embedded` : Được đặt trên một trường trong Entity để chỉ ra rằng trường đó là một đối tượng nhúng. Các thuộc tính của đối tượng nhúng sẽ được ánh xạ trực tiếp vào các cột của bảng chứa Entity.
- `@AttributeOverrides` : Khi một Entity có nhiều trường cùng kiểu `@Embeddable` , bạn cần sử dụng `@AttributeOverrides` để tránh xung đột tên cột bằng cách định nghĩa lại tên cột cho từng trường.

2.5 Annotations cho Inheritance

2.5 Annotations cho Inheritance

JPA hỗ trợ ánh xạ các mối quan hệ kế thừa trong Java sang cơ sở dữ liệu. Annotation

`@Inheritance` được sử dụng để chỉ định chiến lược kế thừa.

2.5.1 @Inheritance

Chỉ định strategy cho inheritance mapping.

Java

```

@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS) // Mỗi lớp con sẽ có
một bảng riêng, bao gồm cả các thuộc tính của lớp cha.
public abstract class Vehicle {

```

```

@Id
@GeneratedValue
private Long id;
private String brand;
// ...
}

@Entity
public class Car extends Vehicle {
    private int numberOfDoors;
    // ...
}

@Entity
public class Motorcycle extends Vehicle {
    private boolean hasSidecar;
    // ...
}

```

Các strategy:

- **SINGLE_TABLE** : Tất cả classes trong hierarchy dùng chung một bảng. Một cột discriminator sẽ được thêm vào để phân biệt các loại Entity con. Đây là chiến lược mặc định và thường hiệu quả nhất về mặt hiệu suất.
- **TABLE_PER_CLASS** : Mỗi concrete class (lớp không trừu tượng) có một bảng riêng. Bảng của lớp con sẽ chứa tất cả các thuộc tính của nó và các thuộc tính kế thừa từ lớp cha. Không có join khi truy vấn.
- **JOINED** : Mỗi class (cả cha và con) có một bảng riêng. Các bảng này được join lại khi truy vấn dữ liệu. Đây là chiến lược chuẩn hóa nhất và thường được ưa chuộng khi có nhiều thuộc tính chung.

Giải thích: Việc lựa chọn chiến lược kế thừa phù hợp phụ thuộc vào yêu cầu về hiệu suất, tính chuẩn hóa và độ phức tạp của mô hình dữ liệu. **SINGLE_TABLE** thường đơn giản và hiệu quả cho các hierarchy nhỏ, trong khi **JOINED** phù hợp cho các mô hình phức tạp hơn.

2.5.2 @DiscriminatorColumn và @DiscriminatorValue

Sử dụng với **SINGLE_TABLE** strategy. Các annotations này giúp JPA biết cách phân biệt các loại Entity con khi sử dụng chiến lược **SINGLE_TABLE**.

Java

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) // Sử dụng một bảng duy
nhất cho toàn bộ cây kế thừa.
@DiscriminatorColumn(name = "vehicle_type", discriminatorType =
DiscriminatorType.STRING) // Định nghĩa một cột mới (vehicle_type) trong bảng
để lưu trữ loại Entity con. Kiểu dữ liệu của cột này là chuỗi.
public abstract class Vehicle {
    // ...
}

@Entity
@DiscriminatorValue("CAR") // Chỉ định giá trị "CAR" cho cột vehicle_type khi
lưu trữ một đối tượng Car.
public class Car extends Vehicle {
    // ...
}

@Entity
@DiscriminatorValue("MOTORCYCLE") // Chỉ định giá trị "MOTORCYCLE" cho cột
vehicle_type khi lưu trữ một đối tượng Motorcycle.
public class Motorcycle extends Vehicle {
    // ...
}
```

Giải thích:

- `@DiscriminatorColumn` : Được sử dụng trên lớp cha để định nghĩa một cột trong bảng `SINGLE_TABLE` sẽ lưu trữ thông tin về loại Entity con. Bạn có thể tùy chỉnh tên cột (`name`) và kiểu dữ liệu của nó (`discriminatorType`).
- `@DiscriminatorValue` : Được sử dụng trên mỗi lớp con để chỉ định giá trị cụ thể sẽ được lưu trữ trong cột discriminator khi một instance của lớp con đó được persist. Điều này giúp JPA biết cách ánh xạ dữ liệu từ bảng chung về đúng loại Entity con khi truy vấn.

2.6 Annotations cho Index và Constraint

2.6.1 @Index

Tạo index cho bảng. Index giúp tăng tốc độ truy vấn dữ liệu.

Java

```

@Entity
@Table(name = "users", indexes = { // Sử dụng thuộc tính indexes trong @Table
    để định nghĩa các index.
    @Index(name = "idx_email", columnList = "email"), // Tạo một index trên
    cột email với tên là idx_email.
    @Index(name = "idx_name_email", columnList = "name, email") // Tạo một
    index tổng hợp trên hai cột name và email.
})
public class User {
    // ...
}

```

Giải thích: Index là một cấu trúc dữ liệu giúp tăng tốc độ tìm kiếm và truy xuất dữ liệu từ bảng. Khi bạn truy vấn dữ liệu dựa trên các cột được đánh index, cơ sở dữ liệu có thể tìm thấy các hàng nhanh hơn. Tuy nhiên, việc tạo quá nhiều index có thể làm chậm các thao tác ghi (INSERT, UPDATE, DELETE) vì cơ sở dữ liệu phải cập nhật cả index.

2.6.2 @UniqueConstraint

Tạo unique constraint. Unique constraint đảm bảo rằng tất cả các giá trị trong một cột hoặc một tập hợp các cột là duy nhất.

Java

```

@Entity
@Table(name = "users", uniqueConstraints = { // Sử dụng thuộc tính
    uniqueConstraints trong @Table để định nghĩa các unique constraint.
    @UniqueConstraint(name = "uk_email", columnNames = "email"), // Đảm bảo
    rằng giá trị trong cột email là duy nhất.
    @UniqueConstraint(name = "uk_username", columnNames = "username") // Đảm
    bảo rằng giá trị trong cột username là duy nhất.
})
public class User {
    // ...
}

```

Giải thích: Unique constraint ngăn chặn việc chèn các hàng có giá trị trùng lặp trong các cột được chỉ định. Điều này giúp duy trì tính toàn vẹn của dữ liệu trong cơ sở dữ liệu. Nếu bạn cố gắng chèn một giá trị đã tồn tại vào một cột có unique constraint, cơ sở dữ liệu sẽ ném ra một ngoại lệ.